Lab 5: Static Code Analysis

Name: Kanishq Surendran **SRN:** PES2UG24CS214 **Repo:** SETAPESU26/L5_26_dairy_collection **File:** dairy_collection.py

Summary: Pylint 2.00/10 → 10.00/10, Bandit 3 issues (2 High, 1 Medium) → 0, Flake8 7 issues → 0. All issues reported by the tools were fixed.

1. Issue Table

Issue	Type	Line(s)	Description	Fix Approach
SQL injection (B608)	Security (Medium)	11	`record_milk` built the INSERT with `.format()`, so a farmer name like `x'); DROP TABLE milk;--` becomes SQL	Used a parameterized query: `VALUES (?, ?, ?, ?)` with a tuple
OS command injection (B605)	Security (High)	31	`os.system("echo ..." + farmer)` runs through a shell, so `;` or `&&` in the name runs extra commands	Removed the shell call, now uses `print(f"printing slip for {farmer}")` and dropped `import os`
Weak MD5 hash (B324)	Security (High)	16	MD5 is broken and unsuitable for identifying tokens	Switched to `hashlib.sha256`
Redefined outer name `conn` (W0621)	Bug risk	6, 10, 19, 24, 34	Global `conn` shadowed by function parameters, which hides mistakes and makes testing hard	Removed the global, added `create_connection()`, `main()` creates `conn` and passes it in
Code ran on import	Design	47	`main()` was called at module level, so importing the file ran the demo	Added `if __name__ == "__main__":` guard
Comparison `fat == None` (C0121 / E711)	Bug risk	36	Equality with `None` is wrong style and can misbehave with custom `__eq__`	Changed to `fat is None`
Ambiguous name `l` (E741)	Style	26	Single letter `l` looks like `1` or `I`	Renamed to `best`
Unused import `json` (W0611 / F401)	Style	2	Imported but never used	Removed it
Function names not snake_case (C0103)	Style	10, 15, 19, 24, 30, 34	`recordMilk`, `farmerToken` and others break PEP 8	Renamed to `record_milk`, `farmer_token`, `daily_total`, `top_farmer`, `print_slip`, `low_fat`
Missing docstrings (C0114 / C0116)	Style	1, all fun ctions	No module or function docs	Added a docstring to the module and every function
Lines too long (C0301 / E501)	Style	7, 11, 20, 25	Lines went past 79 characters	Wrapped long strings and queries over multiple lines
Old-style formatting (C0209)	Style	11, 21	`.format()` and `%` used instead of f-strings	Used f-strings (`f"Total: {total:.1f} litres"`)
No input validation	Bug	10	Empty farmer or day, or negative litres or fat, went straight into the database	Added checks in `record_milk` that raise `ValueError`
Crash on empty table	Bug	26	`max()` on an empty list raised `ValueError` in `top_farmer`	Return `None` when there are no rows
Magic number `3.5`	Style	36	The low-fat limit was hard-coded	Moved to the constant `LOW_FAT_LIMIT`

2. Cleaned Code (dairy_collection.py)

```
"""Dairy cooperative milk collection ledger.

Records each farmer's milk deliveries, totals a day's collection, finds the
top supplier, flags low-fat milk and prints collection slips.
"""
import hashlib
import sqlite3

LOW_FAT_LIMIT = 3.5


def create_connection():
    """Create an in-memory database with the milk table."""
    conn = sqlite3.connect(":memory:")
    conn.execute(
        "CREATE TABLE milk ("
        "id INTEGER PRIMARY KEY, farmer TEXT, litres REAL, "
        "fat REAL, day TEXT)"
    )
    return conn


def record_milk(conn, farmer, litres, fat, day):
    """Store one milk delivery after validating the input."""
    if not farmer or not day:
        raise ValueError("farmer and day must not be empty")
    if litres < 0 or fat < 0:
        raise ValueError("litres and fat must not be negative")
    conn.execute(
        "INSERT INTO milk (farmer, litres, fat, day) VALUES (?, ?, ?, ?)",
        (farmer, litres, fat, day),
    )
    conn.commit()


def farmer_token(farmer):
    """Return a SHA-256 token identifying the farmer."""
    return hashlib.sha256(farmer.encode()).hexdigest()


def daily_total(conn, day):
    """Return a printable total of litres collected on the given day."""
    rows = conn.execute(
        "SELECT litres FROM milk WHERE day = ?", (day,)
    ).fetchall()
    total = sum(row[0] for row in rows)
    return f"Total: {total:.1f} litres"


def top_farmer(conn):
    """Return the farmer who supplied the most litres, or None if empty."""
    rows = conn.execute(
        "SELECT farmer, SUM(litres) FROM milk GROUP BY farmer"
    ).fetchall()
    if not rows:
        return None
    best = max(rows, key=lambda row: row[1])
    return best[0]


def print_slip(farmer):
    """Print a collection slip for the farmer."""
    print(f"printing slip for {farmer}")


def low_fat(conn):
    """Return farmers whose milk has missing or low fat content."""
    rows = conn.execute("SELECT farmer, fat FROM milk").fetchall()
    return [
        farmer for farmer, fat in rows
        if fat is None or fat < LOW_FAT_LIMIT
    ]


def main():
    """Run a short demonstration of the ledger."""
    conn = create_connection()
    record_milk(conn, "Gopal", 12.5, 4.1, "Mon")
    record_milk(conn, "Lata", 20.0, 3.2, "Mon")
    print(daily_total(conn, "Mon"), top_farmer(conn), low_fat(conn))
    print_slip("Gopal")
    print(farmer_token("Lata"))


if __name__ == "__main__":
    main()
```

3. Tool Reports — Before Fixes

Pylint

```
************* Module dairy_collection
dairy_collection.py:7:0: C0301: Line too long (104/100) (line-too-long)
dairy_collection.py:11:0: C0301: Line too long (124/100) (line-too-long)
dairy_collection.py:1:0: C0114: Missing module docstring (missing-module-docstring)
dairy_collection.py:10:0: C0116: Missing function or method docstring (missing-function-docstring)
dairy_collection.py:10:0: C0103: Function name "recordMilk" doesn't conform to snake_case naming style (invalid-name)
dairy_collection.py:10:15: W0621: Redefining name 'conn' from outer scope (line 6) (redefined-outer-name)
dairy_collection.py:11:17: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
dairy_collection.py:15:0: C0116: Missing function or method docstring (missing-function-docstring)
dairy_collection.py:15:0: C0103: Function name "farmerToken" doesn't conform to snake_case naming style (invalid-name)
dairy_collection.py:19:0: C0116: Missing function or method docstring (missing-function-docstring)
dairy_collection.py:19:0: C0103: Function name "dailyTotal" doesn't conform to snake_case naming style (invalid-name)
dairy_collection.py:19:15: W0621: Redefining name 'conn' from outer scope (line 6) (redefined-outer-name)
dairy_collection.py:21:11: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
dairy_collection.py:24:0: C0116: Missing function or method docstring (missing-function-docstring)
dairy_collection.py:24:0: C0103: Function name "topFarmer" doesn't conform to snake_case naming style (invalid-name)
dairy_collection.py:24:14: W0621: Redefining name 'conn' from outer scope (line 6) (redefined-outer-name)
dairy_collection.py:30:0: C0116: Missing function or method docstring (missing-function-docstring)
dairy_collection.py:30:0: C0103: Function name "printSlip" doesn't conform to snake_case naming style (invalid-name)
dairy_collection.py:34:0: C0116: Missing function or method docstring (missing-function-docstring)
dairy_collection.py:34:0: C0103: Function name "lowFat" doesn't conform to snake_case naming style (invalid-name)
dairy_collection.py:34:11: W0621: Redefining name 'conn' from outer scope (line 6) (redefined-outer-name)
dairy_collection.py:36:36: C0121: Comparison 'fat == None' should be 'fat is None' (singleton-comparison)
dairy_collection.py:39:0: C0116: Missing function or method docstring (missing-function-docstring)
dairy_collection.py:2:0: W0611: Unused import json (unused-import)

-----------------------------------
Your code has been rated at 2.00/10
```

Flake8

```
dairy_collection.py:2:1: F401 'json' imported but unused
dairy_collection.py:7:80: E501 line too long (104 > 79 characters)
dairy_collection.py:11:80: E501 line too long (124 > 79 characters)
dairy_collection.py:20:80: E501 line too long (83 > 79 characters)
dairy_collection.py:25:80: E501 line too long (90 > 79 characters)
dairy_collection.py:26:5: E741 ambiguous variable name 'l'
dairy_collection.py:36:41: E711 comparison to None should be 'if cond is None:'
```

Bandit

```
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.16
Run started:2026-10-14 04:13:42.039483+00:00

Test results:
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
   Severity: Medium   Confidence: Medium
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
   Location: ./dairy_collection.py:11:17
10  def recordMilk(conn, farmer, litres, fat, day):
11      conn.execute("INSERT INTO milk (farmer, litres, fat, day) VALUES ('{}', {}, {}, '{}')".format(farmer, litres, fat, day))
12      conn.commit()

--------------------------------------------------
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
   Severity: High   Confidence: High
   CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
   Location: ./dairy_collection.py:16:11
15  def farmerToken(farmer):
16      return hashlib.md5(farmer.encode()).hexdigest()
17

--------------------------------------------------
>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible injection detected, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b605_start_process_with_a_shell.html
   Location: ./dairy_collection.py:31:4
30  def printSlip(farmer):
31      os.system("echo printing slip for " + farmer)
32

--------------------------------------------------

Code scanned:
    Total lines of code: 30
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0
Run metrics:
```

```
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 1
        High: 2
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 1
        High: 2
Files skipped (0):
```

4. Tool Reports — After Fixes

Pylint

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 2.00/10, +8.00)
```

Flake8

```
(no output: no issues found)
```

Bandit

```
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.16
Run started:2026-10-14 04:14:03.464781+00:00

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 66
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
Files skipped (0):
```

5. Reflection

1. Which issues were the easiest and hardest to fix?
Easiest were the style ones (unused import, naming, docstrings, `is None`), since the tool names the line and the fix is mechanical. Hardest were the security ones (SQL injection and shell injection), because they need a change in how the code works. Even then the fix was small: `?` placeholders for SQL, and removing the shell call entirely.

2. Did the tools report any false positives?
Mostly no. Every Bandit finding was a real risk. The MD5 warning is the closest to debatable, since MD5 is harmless for a non-security checksum. Here the hash is used as a farmer token, so I switched to SHA-256 anyway. Pylint's "line too long (100)" and Flake8's "79" disagree on the limit, so one tool can complain when the other doesn't. That is a config difference, not a bug.

3. How would you integrate static analysis into your workflow?
Run Flake8 and Pylint in the editor and as a pre-commit hook, so problems are caught before a commit. Run all three tools in CI (for example GitHub Actions) on every pull request and block the merge if Bandit reports High issues or the Pylint score drops. Keep one shared config file in the repo so everyone uses the same rules.

4. What improvements did you see in code quality, readability and security?
Pylint went from 2.00 to 10.00, and Bandit and Flake8 went from 10 findings to 0. The code is easier to read (snake_case names, docstrings, short lines, f-strings). It is also safer: no injection holes, no weak hash. It is more robust too: input is validated, `top_farmer` no longer crashes on an empty table, and importing the file no longer runs the demo.